

Deploying a React Website on AWS with HTTPS

S3 + CloudFront + ACM + Route 53, using a domain bought on GoDaddy

LIVE RESULT

<https://shopvra.space>

Also: <https://www.shopvra.space> · Free CloudFront plan · Free ACM certificate
CloudFront distribution E1P27P6jXO8L71 · Region us-east-1

INSIDE THIS GUIDE

- 1 Ways to host a static website on AWS
- 2 What we built (architecture + resource list)
- 3 Step-by-step setup, with screenshots (13 steps)
- 4 Security clean-up for the S3 bucket
- 5 Deploying future updates
- 6 Troubleshooting
- 7 Next: the backend at <https://api.shopvra.space>

SECTION 1

Ways to host a static website on AWS

A React app built with `npm run build` is just HTML, CSS, JS and images. AWS gives you several ways to serve those files. They differ mainly in HTTPS support, effort and cost.

Option	HTTPS	Custom domain	Effort	Typical cost	Best for
S3 static website endpoint	No (HTTP only)	HTTP only	Very easy	Pennies	Quick tests, demos
S3 + CloudFront (used here)	Yes	Yes (ACM)	Medium	Free plan / low	Production sites, full control
AWS Amplify Hosting	Yes	Yes (auto)	Easiest	Low, pay per build + traffic	Git-based auto deploys
Amazon Lightsail (bucket + CDN)	Yes	Yes	Easy	Fixed monthly bundle	Simple predictable pricing
EC2 + nginx + Let's Encrypt	Yes	Yes	High	Instance cost 24/7	When you also need a server

How to choose

- **S3 website endpoint alone** is what you had first (`http://ecommerce-web-app-...s3-website-us-east-1.amazonaws.com`). It works, but it can never serve HTTPS and the bucket must be public.
- **S3 + CloudFront** adds HTTPS, a global CDN, caching, your own domain and a **private** bucket. This is the standard production setup and what this guide covers.
- **Amplify** does the CloudFront + certificate work for you and redeploys on every Git push. You used it for the job app. It's the fastest route, with less control.
- **EC2 / Lightsail instances** only make sense if you also need to run server code on the same machine.

Where should DNS live?

Approach	Root domain (shopvra.space)	Notes
Move nameservers to Route 53 (used here)	Supported via Alias records	≈ \$0.50/month per hosted zone. ACM and CloudFront can create records for you automatically.
Keep DNS at GoDaddy	Not with a CNAME; use GoDaddy forwarding to www	Add CNAMEs for www / api manually. Works, but more manual steps.

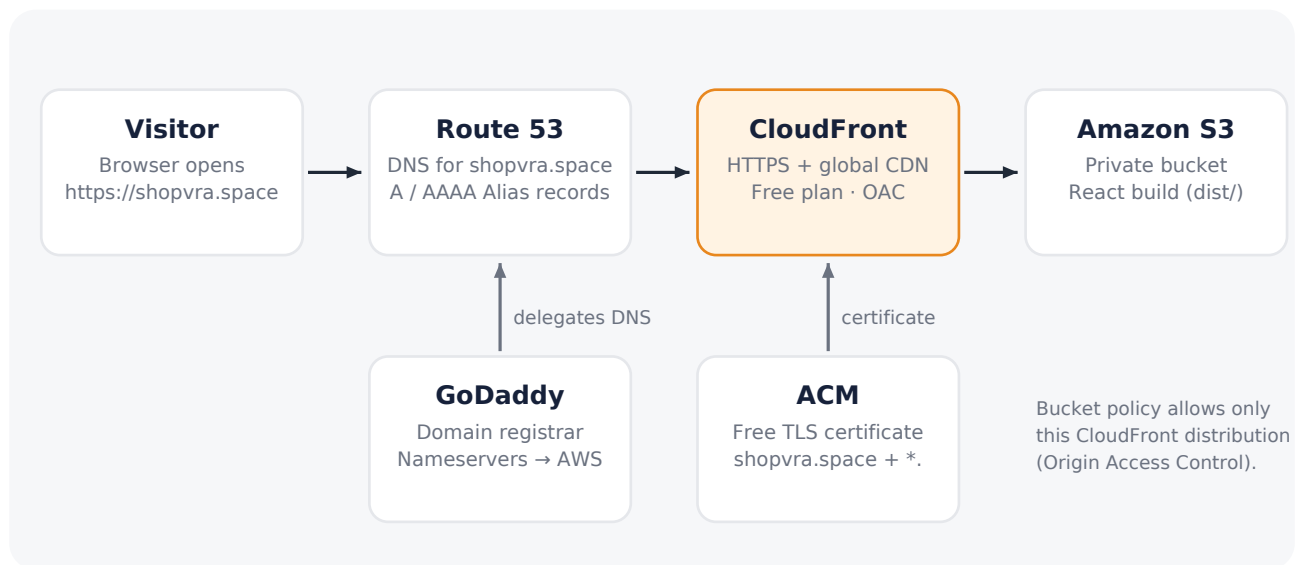
Good to know

The domain stays registered (and renewed) at GoDaddy. Only the **DNS hosting** moved to Route 53 by changing the nameservers.

SECTION 2

What we built

The final architecture and the AWS resources it uses.



How a request flows: the visitor's browser asks Route 53 where **shopvra.space** lives, Route 53 answers with CloudFront, CloudFront serves the site over HTTPS using the ACM certificate, and fetches files from the private S3 bucket when its cache is empty.

Resource reference

Item	Value
Domain (registrar)	shopvra.space (GoDaddy)
Route 53 hosted zone	Z03554922XU0BK1VBC830
Nameservers	ns-1579.awsdns-05.co.uk · ns-647.awsdns-16.net ns-110.awsdns-13.com · ns-1377.awsdns-44.org
ACM certificate (us-east-1)	550d0388-ba3b-4dce-87f9-fefb9e86b181 covers shopvra.space and *.shopvra.space
S3 bucket	ecommerce-web-app-084767318696 (us-east-1)
CloudFront distribution	E1P27P6JX08L71 · d2mxs46pccx59q.cloudfront.net
CloudFront plan	Flat-rate Free plan (\$0/month)
Live URLs	https://shopvra.space · https://www.shopvra.space

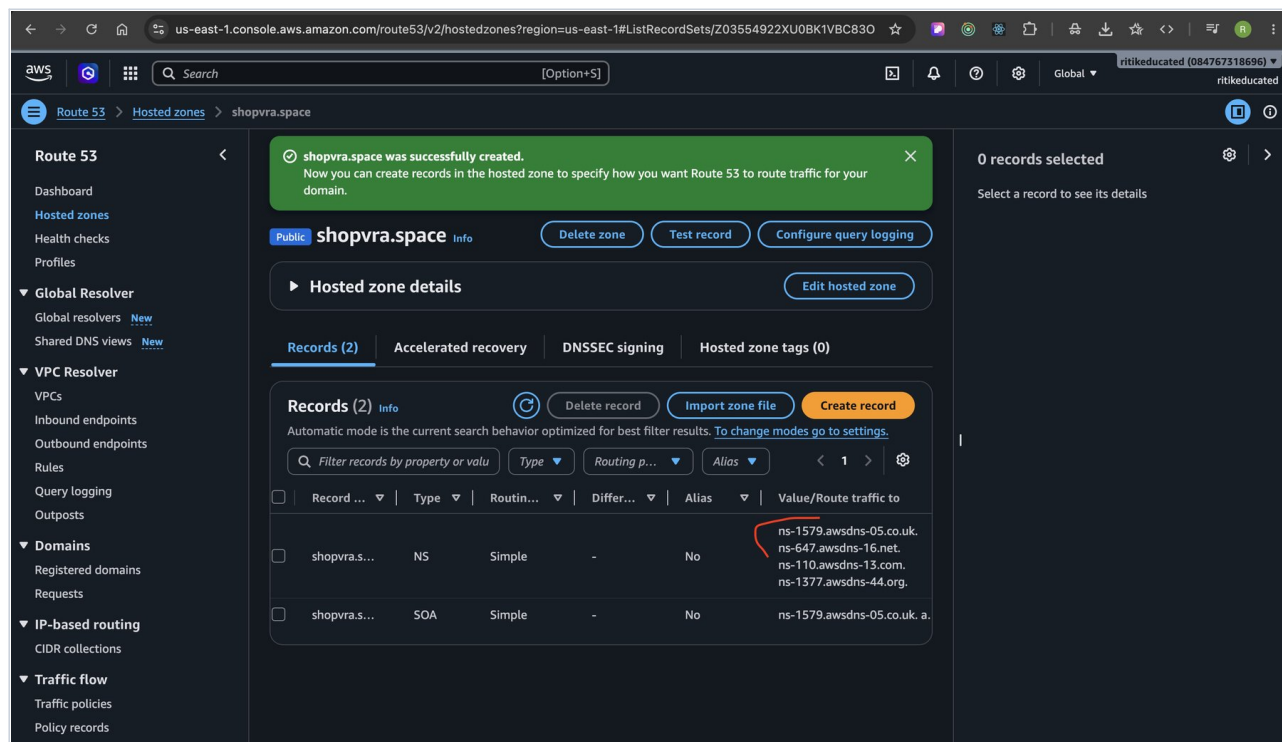
SECTION 3

Step-by-step setup

Exactly what was done, in order, with the screenshots from the real setup. Prerequisite: the React build was already uploaded to the S3 bucket.

1 Step 1 — Create a hosted zone in Route 53

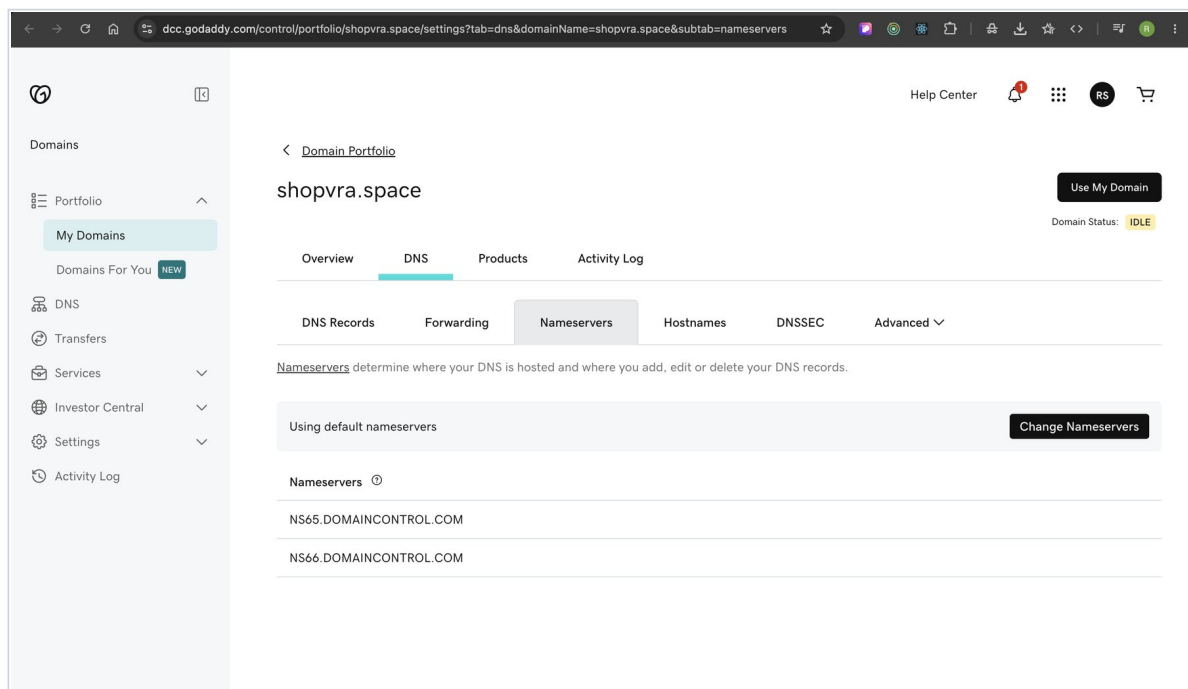
Route 53 → **Hosted zones** → **Create hosted zone** → enter **shopvra.space** → Public hosted zone. AWS creates an NS and an SOA record. The four NS values (circled) are what GoDaddy needs.



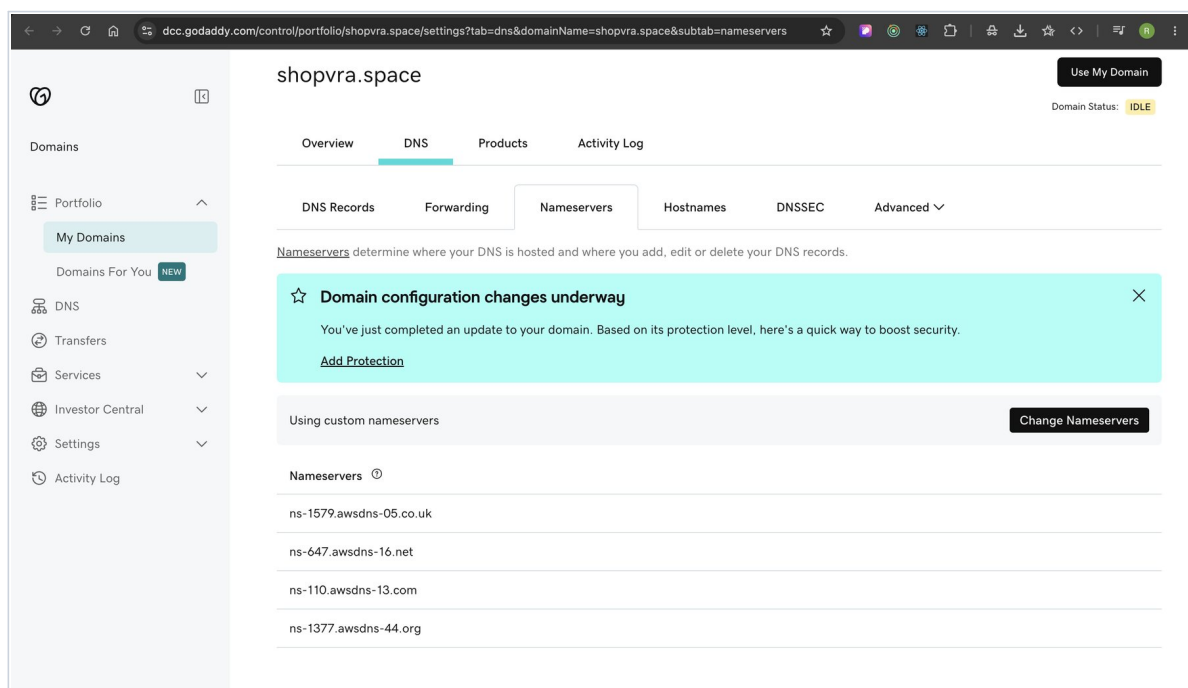
Route 53 hosted zone for shopvra.space. Copy the four nameserver values from the NS record.

2 Step 2 — Point GoDaddy at the Route 53 nameservers

GoDaddy → **My Domains** → shopvra.space → **DNS** → **Nameservers** → **Change Nameservers** → "I'll use my own nameservers". Remove the two DOMAINCONTROL.COM entries and add the four AWS ones (without a trailing dot).



Before: GoDaddy's default nameservers.



After: "Using custom nameservers" with the four AWS values. Changes take from 30 minutes to a few hours to spread.

Heads-up

Once nameservers point to Route 53, any DNS records that existed at GoDaddy (for example email MX records) stop working and must be re-created in Route 53.

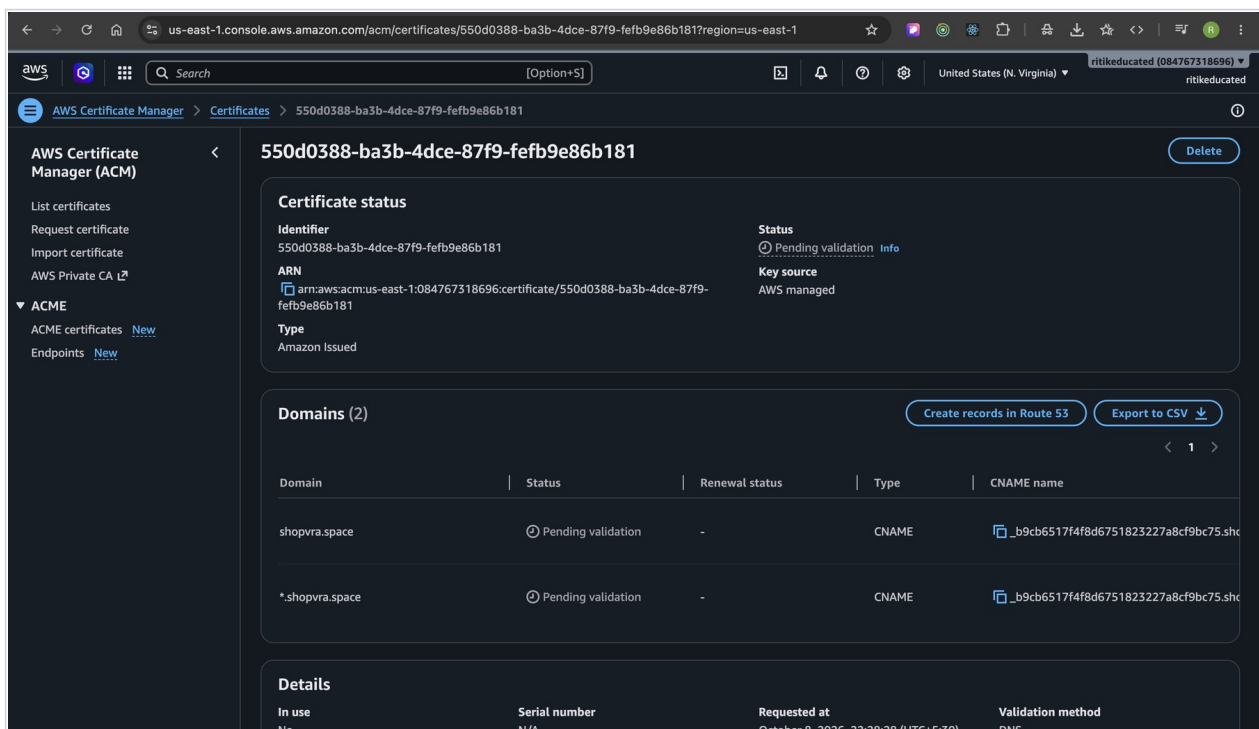
3 Step 3 — Request a free SSL/TLS certificate (ACM)

Switch the region to **US East (N. Virginia) us-east-1**. CloudFront only uses certificates from this region. Certificate Manager → **Request** → **Request a public certificate**.

- Domain names: **shopvra.space** and ***.shopvra.space** (the wildcard covers www, api and future subdomains)
- Allow export: **Disable export** (keeps it free)
- Validation method: **DNS validation**
- Key algorithm: RSA 2048 (default) → **Request**

The screenshot shows the AWS Certificate Manager console in the us-east-1 region. The breadcrumb trail is: AWS Certificate Manager > Certificates > Request certificate > Request public certificate. The main heading is 'Request public certificate'. Below it, the 'Domain names' section asks to provide one or more domain names. Two input fields are shown: 'shopvra.space' and '*.shopvra.space', each with a 'Remove' button. An 'Add another name to this certificate' button is also present. Below this, the 'Allow export' section has two radio buttons: 'Disable export' (selected) and 'Enable export'. The 'Validation method' section has two radio buttons: 'DNS validation' (selected) and 'Email validation - deprecated'. The console interface is dark-themed.

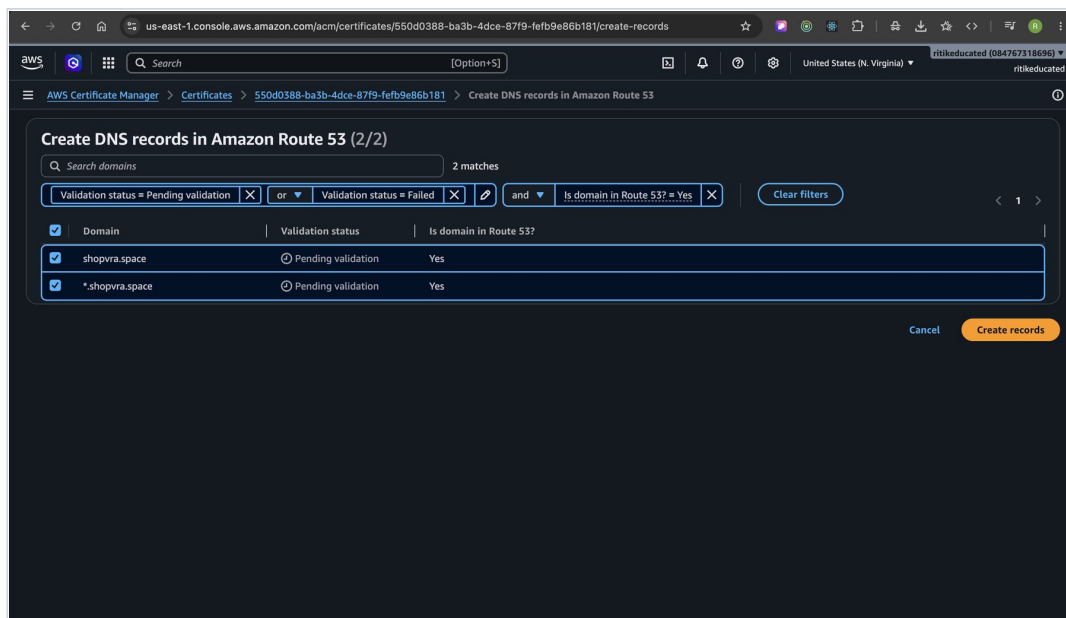
Requesting the public certificate for the root domain and the wildcard.



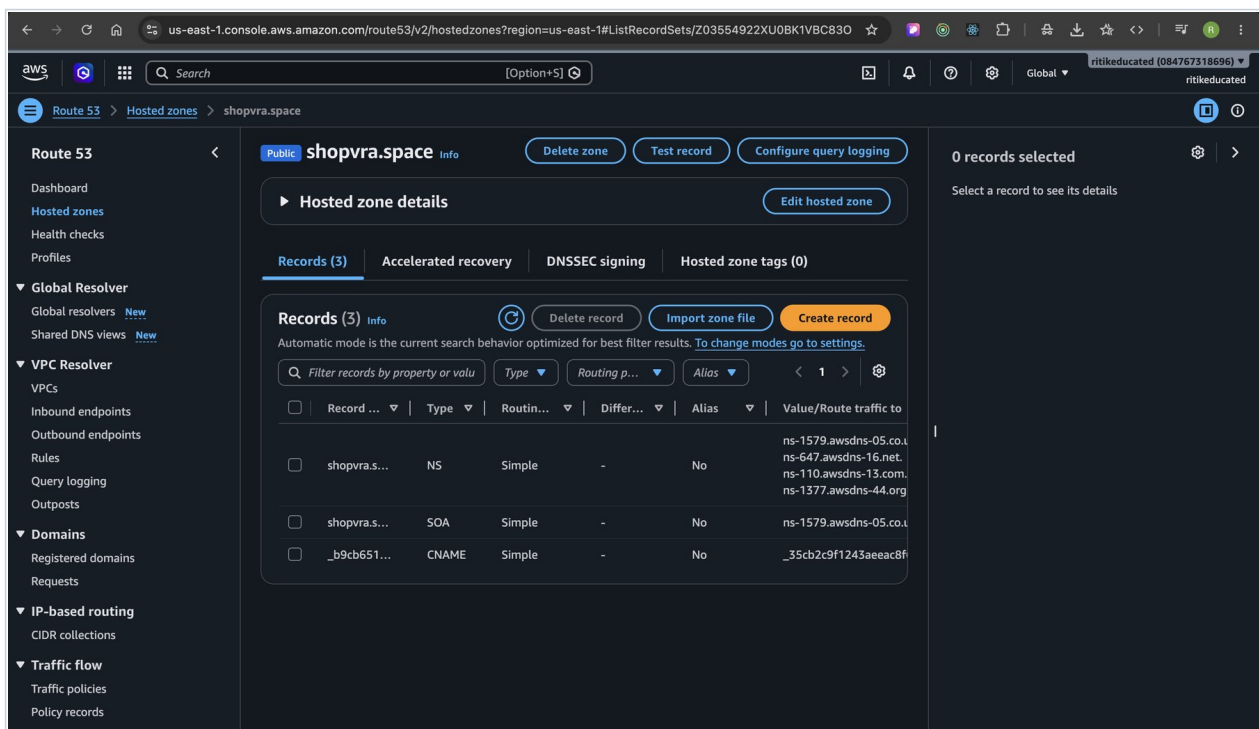
The certificate starts in “Pending validation”. AWS now needs proof that you own the domain.

4 Step 4 — Validate the certificate through Route 53

On the certificate page, click **Create records in Route 53**. Both domains share one CNAME, so AWS adds a single record.



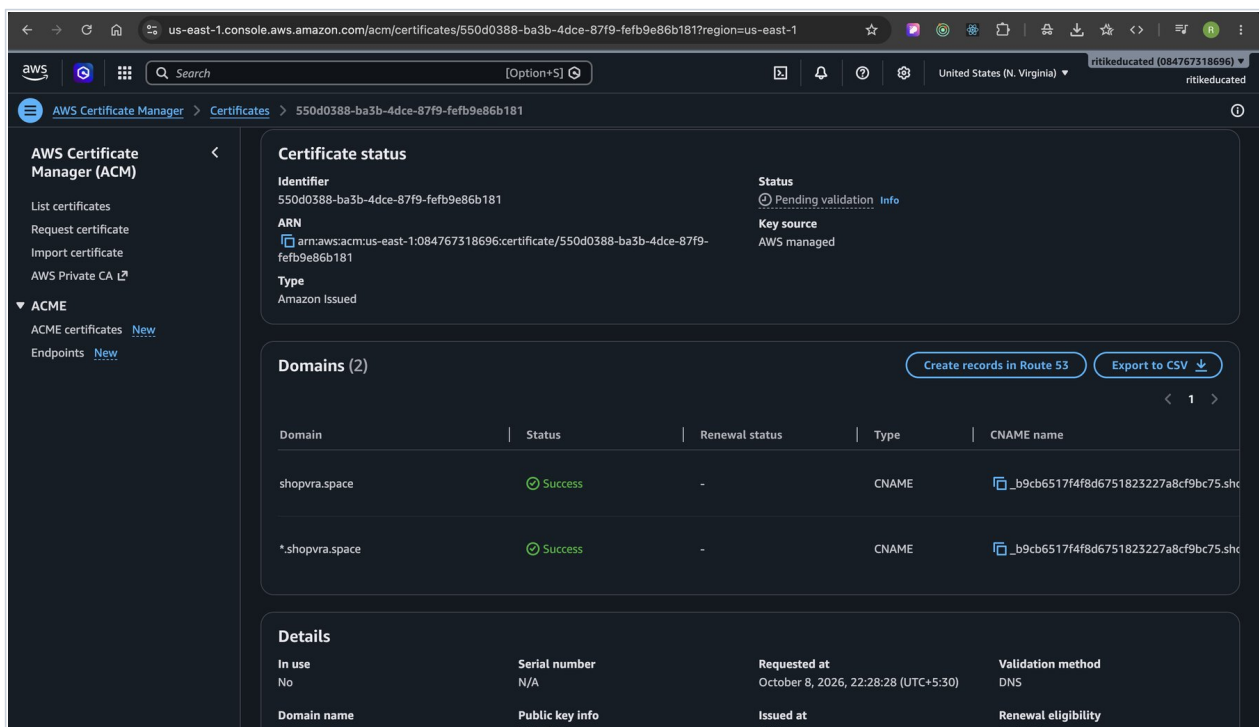
Both domains selected, “Is domain in Route 53?” = Yes → Create records.



The screenshot shows the AWS Route 53 console for the hosted zone `shopvra.space`. The left sidebar contains navigation links for Route 53, Global Resolver, VPC Resolver, Domains, and IP-based routing. The main content area displays the hosted zone details, including buttons for `Delete zone`, `Test record`, and `Configure query logging`. The `Records (3)` tab is active, showing a table of records:

Record	Type	Routing	Differ...	Alias	Value/Route traffic to
shopvra.s...	NS	Simple	-	No	ns-1579.awsdns-05.co.t ns-647.awsdns-16.net. ns-110.awsdns-13.com. ns-1377.awsdns-44.org
shopvra.s...	SOA	Simple	-	No	ns-1579.awsdns-05.co.t
_b9cb651...	CNAME	Simple	-	No	_35cb2c9f1243aeac8f

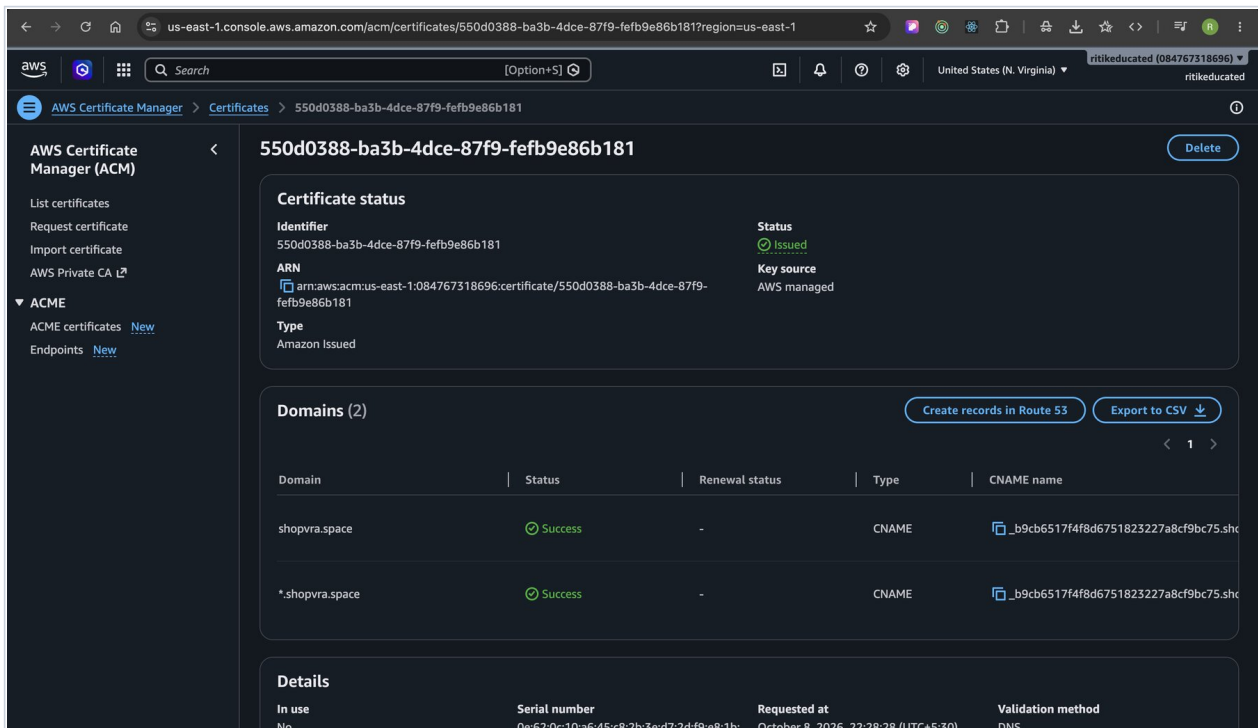
The hosted zone now has a third record: the ACM validation CNAME starting with `_b9cb651...`



The screenshot shows the AWS Certificate Manager console for the certificate `550d0388-ba3b-4dce-87f9-feb9e86b181`. The left sidebar contains navigation links for AWS Certificate Manager (ACM) and ACME. The main content area displays the certificate status, including buttons for `Create records in Route 53` and `Export to CSV`. The `Domains (2)` section shows a table of domains:

Domain	Status	Renewal status	Type	CNAME name
shopvra.space	Success	-	CNAME	_b9cb6517f4f8d6751823227a8cf9bc75.shc
*.shopvra.space	Success	-	CNAME	_b9cb6517f4f8d6751823227a8cf9bc75.shc

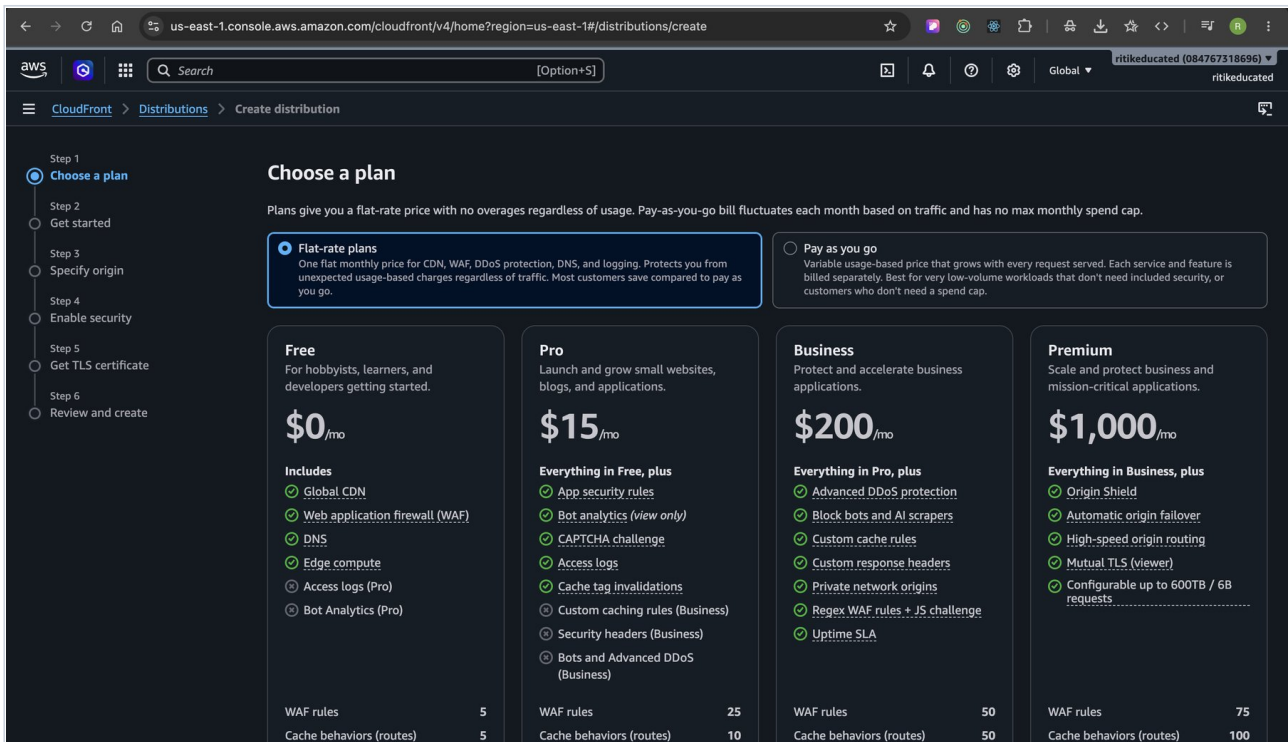
After DNS spreads, each domain shows "Success". The overall status follows a few minutes later.



Status: Issued. The certificate is ready to attach to CloudFront.

5 Step 5 — Create the CloudFront distribution: choose a plan

CloudFront → **Create distribution**. The new console starts with a plan picker. Pick **Flat-rate plans** → **Free (\$0/mo)**. It includes the CDN, HTTPS, basic WAF and DNS, with monthly usage allowances that are plenty for a personal or portfolio site. Avoid **Pay as you go** for now because it has no spending cap.



The plan picker. Free is enough here, and you can upgrade later from Manage plan.

6 Step 6 — Get started: name and domain

- Distribution name: **shopvra-frontend**; type: **Single website configuration**
- Route 53 managed domain: **shopvra.space** → **Check domain** (it detects the hosted zone)
- Click **Add a subdomain** → **www**, then Next

The screenshot shows the AWS CloudFront console interface for creating a new distribution. On the left, a sidebar lists steps from 'Get started' to 'Review and create'. The main area is titled 'Create distribution' and contains two main sections: 'Distribution options' and 'Domain'. In the 'Distribution options' section, the 'Distribution name' is 'shopvra-frontend' and the 'Distribution type' is 'Single website configuration'. The 'Domain' section shows 'shopvra.space' and a 'Check domain' button. A green message box states: 'Domain managed by Route 53. We found a Route 53 zone file in your account for shopvra.space. CloudFront can automatically manage your DNS for you.' At the bottom, there is an 'Add a subdomain' button.

CloudFront finds the Route 53 zone and can manage the DNS records for you.

7 Step 7 — Specify the origin (the S3 bucket)

- Origin type: **Amazon S3**
- S3 origin: `ecommerce-web-app-084767318696.s3.us-east-1.amazonaws.com` (the bucket/REST endpoint)
- **Do not** click “Use website endpoint”. That endpoint is HTTP-only and forces the bucket to stay public.
- Origin path: leave empty. Settings: **recommended** (private access through Origin Access Control)

The screenshot shows the 'Specify origin' step in the AWS CloudFront console. On the left, a progress bar indicates the current step is 'Specify origin'. The main content area is titled 'Specify origin' and contains two sections: 'Origin type' and 'Origin'. In the 'Origin type' section, 'Amazon S3' is selected. In the 'Origin' section, the 'S3 origin' is configured with the domain 'ecommerce-web-app-084767318696.s3.us-east-1.amazonaws.com'. A yellow warning box states: 'This S3 bucket has static web hosting enabled. If you plan to use this distribution as a website, we recommend using the S3 website endpoint rather than the bucket endpoint.' Below this, the 'Origin path - optional' field is set to '/path'.

Keep the bucket endpoint and ignore the yellow website-endpoint suggestion.

8 Step 8 — Attach the TLS certificate, then review and create

On **Get TLS certificate**, choose the issued **shopvra.space** certificate. It covers both names. Leave the security step on its defaults, review, and click **Create distribution**.

The screenshot shows the 'Get TLS certificate' step in the AWS CloudFront console. On the left, the progress bar indicates the current step is 'Get TLS certificate'. The main content area is titled 'Get TLS certificate' and contains two sections: 'TLS certificate' and 'Certificate details'. In the 'TLS certificate' section, the 'shopvra.space' certificate is selected. In the 'Certificate details' section, the 'ARN' is 'arn:aws:acm:us-east-1:084767318696:certificate/550d0388-ba3b-4dce-87f9-fefb9e86b181', the 'Covered domains' are 'shopvra.space' and '*.shopvra.space', and the 'Source' is 'Amazon'.

The ACM certificate from Steps 3-4 appears automatically because it is issued and in us-east-1.

Successfully created new distribution.

shopvra-frontend Free plan View metrics

Details

- Distribution domain name**: d2mxs46pcx59q.cloudfront.net
- Billing**: Free plan (\$0/month) Manage plan
- ARN**: arn:aws:cloudfront:084767318696:distribution/E1P27P6JXO8L71
- Last modified**: Deploying

General | Security | Origins | Behaviors | Error pages | Invalidations | Logging | Tags

Settings Edit

- Name**: shopvra-frontend
- Description**: Single website or app
- Price class**: Use all edge locations (best performance)
- Supported HTTP versions**: HTTP/2, HTTP/1.1, HTTP/1.0
- Alternate domain names**:
 - shopvra.space
 - www.shopvra.spaceRoute domains to CloudFront
- Custom SSL certificate**: shopvra.space
- Security policy**: TLSv1.2_2021
- Standard logging**: Available with the Pro plan
- Default root object**: -

Distribution created: Free plan, both alternate domain names, and the custom SSL certificate attached.

9 Step 9 — Route the domain to CloudFront

On the distribution's General tab, click **Route domains to CloudFront** → **Set up routing automatically**. This writes A (IPv4) and AAAA (IPv6) Alias records for both names in Route 53.

Successfully updated distribution settings.

shopvra-frontend Free plan View metrics

Details

- Distribution domain name**: d2mxs46pcx59q.cloudfront.net

General | Security | Origins

Settings

- Name**: shopvra-frontend
- Description**: Single website or app
- Price class**: Use all edge locations (best performance)
- Supported HTTP versions**: HTTP/2, HTTP/1.1, HTTP/1.0
- Custom SSL certificate**: shopvra.space
- Security policy**: TLSv1.2_2021

Set up DNS routing

CloudFront can update your DNS records in Route 53 to route traffic from the specified domains to this distribution. If you have configured DNS for these domains otherwise, this may overwrite your changes. You can view the fields that CloudFront will update, and make changes yourself if necessary.

[View the onboarding guide for more information about how to set up DNS for CloudFront](#)

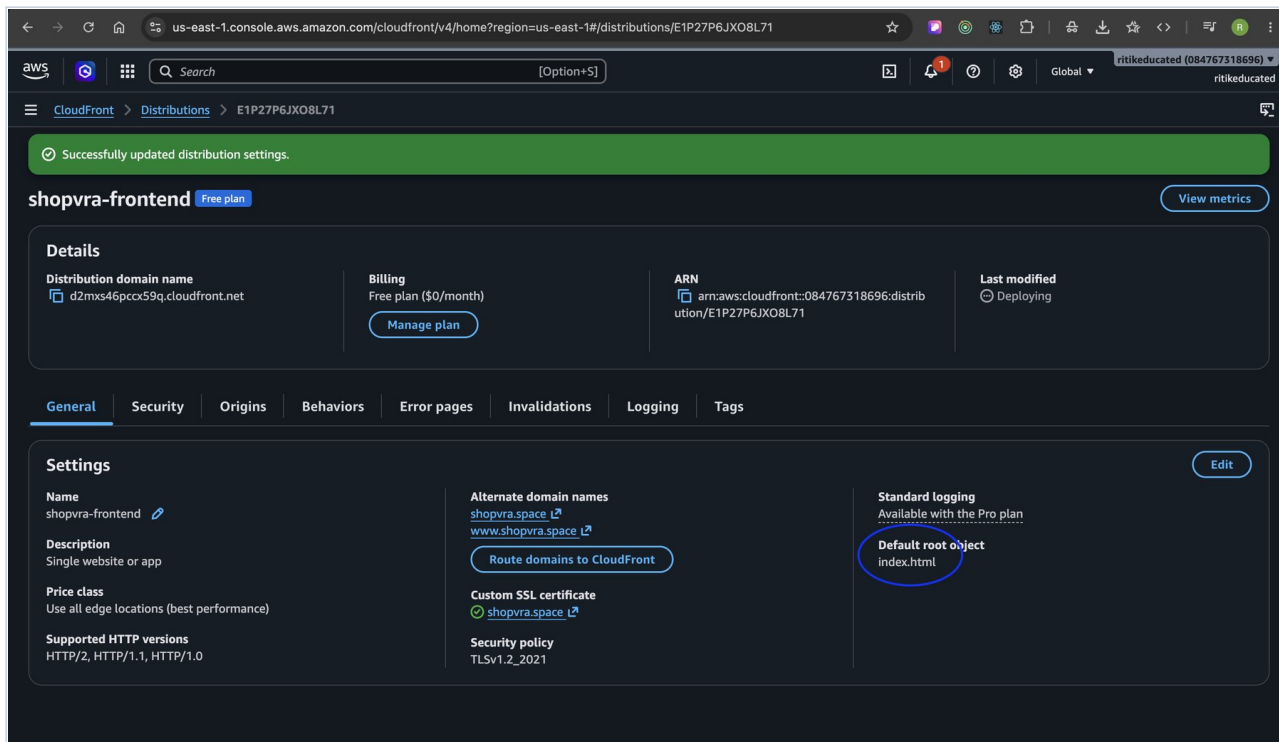
Record type	Name	Value
A	shopvra.s...	d2mxs46pcx59q.cloudfront.net
AAAA	shopvra.s...	d2mxs46pcx59q.cloudfront.net
A	www.shop...	d2mxs46pcx59q.cloudfront.net
AAAA	www.shop...	d2mxs46pcx59q.cloudfront.net

Cancel Set up routing automatically

CloudFront creates four Route 53 records pointing to d2mxs46pcx59q.cloudfront.net.

10 Step 10 — Set the default root object

General → Settings → **Edit** → Default root object: **index.html** → Save. Without it, opening <https://shopvra.space> (the root URL) returns an error instead of the app.



Default root object now set to `index.html`.

11 Step 11 — Add error pages so React Router works

React Router handles routes like `/collection` in the browser, but S3 has no file at that path, so a refresh returns 403/404. Error pages → **Create custom error response**, twice:

HTTP error code	Customize response	Response page path	HTTP response code
403	Yes	<code>/index.html</code>	200
404	Yes	<code>/index.html</code>	200

us-east-1.console.aws.amazon.com/cloudfront/v4/home?region=us-east-1#/distri...

aws [Search] [Option+S] Global ritikeducatd (084767318696) ritikeducatd

CloudFront > Distributions > E1P27P6JXO8L71

Successfully created 404 error page.

shopvra-frontend [Free plan](#) [View metrics](#)

Details

Distribution domain name d2mxs46pccx59q.cloudfront.net	Billing Free plan (\$0/month) Manage plan	ARN arn:aws:cloudfront::084767318696:distribution/E1P27P6JXO8L71	Last modified Deploying
--	--	--	-----------------------------------

General **Security** **Origins** **Behaviors** **Error pages** **Invalidations** **Logging** **Tags**

Error pages (2) [Edit](#) [Delete](#) [Create custom error response](#)

	HTTP error code	Minimum TTL (seconds)	Response page path	HTTP response code
☐	403	10	/index.html	200
☐	404	10	/index.html	200

Both custom error responses in place. Deep links and page refreshes now load the app.

12 Step 12 — Allow CloudFront to read the bucket (OAC bucket policy)

Origins tab → select the origin → **Edit**. With **Origin access control settings** selected, click **Copy policy**, then **Go to S3 bucket permissions** → Bucket policy → Edit → paste → Save.

us-east-1.console.aws.amazon.com/cloudfront/v4/home?region=us-east-1#/distributions/E1P27P6JXO8L71/origins/edit/ecommer...

aws [Search] [Option+S] Global ritikeducatd (084767318696) ritikeducatd

CloudFront > Distributions > E1P27P6JXO8L71 > Edit origin

Enter a name for this origin.
ecommerce-web-app-084767318696.s3.us-east-1.amazonaws.com-muzsvpio4d

Origin access **Info**

☐ Public
Bucket must allow public access.

☒ **Origin access control settings (recommended)**
Bucket can restrict access to only CloudFront.

☐ Legacy access identities
Use a CloudFront origin access identity (OAI) to access the S3 bucket.

Origin access control
Select an existing origin access control (recommended) or create a new control.

oac-ecommerce-web-app-084767318696.s3.us-east-1.amazonaws.com-muzsvpio4d [Create new OAC](#)

[Go to S3 bucket permissions](#)

☐ **Enable origin mutual TLS** Available with the Business plan
Securely validate the connection between your origin and the CloudFront distribution.

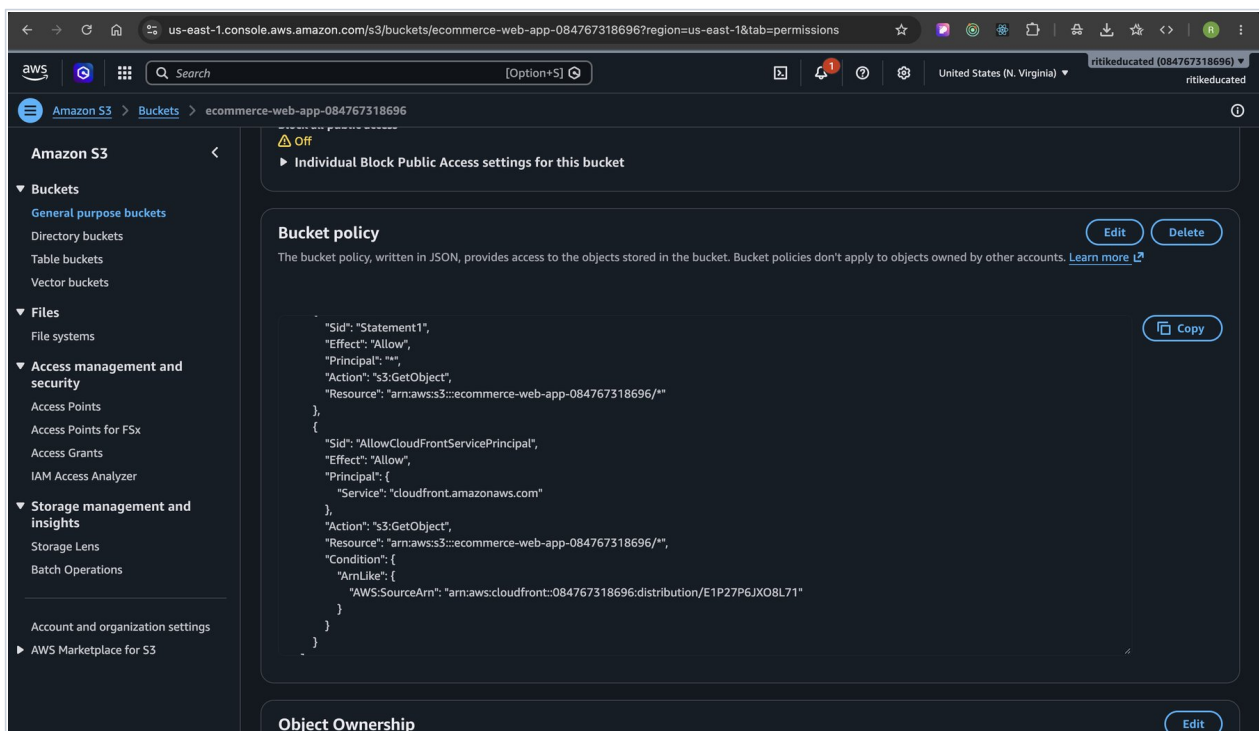
Add custom header - optional
CloudFront includes this header in all requests that it sends to your origin.

[Add header](#)

☐ **Origin Shield** Available with the Premium plan [Upgrade](#)
Maximize performance and reduce origin costs by consolidating duplicate requests and routing them through the fastest path to your origin.

Additional settings

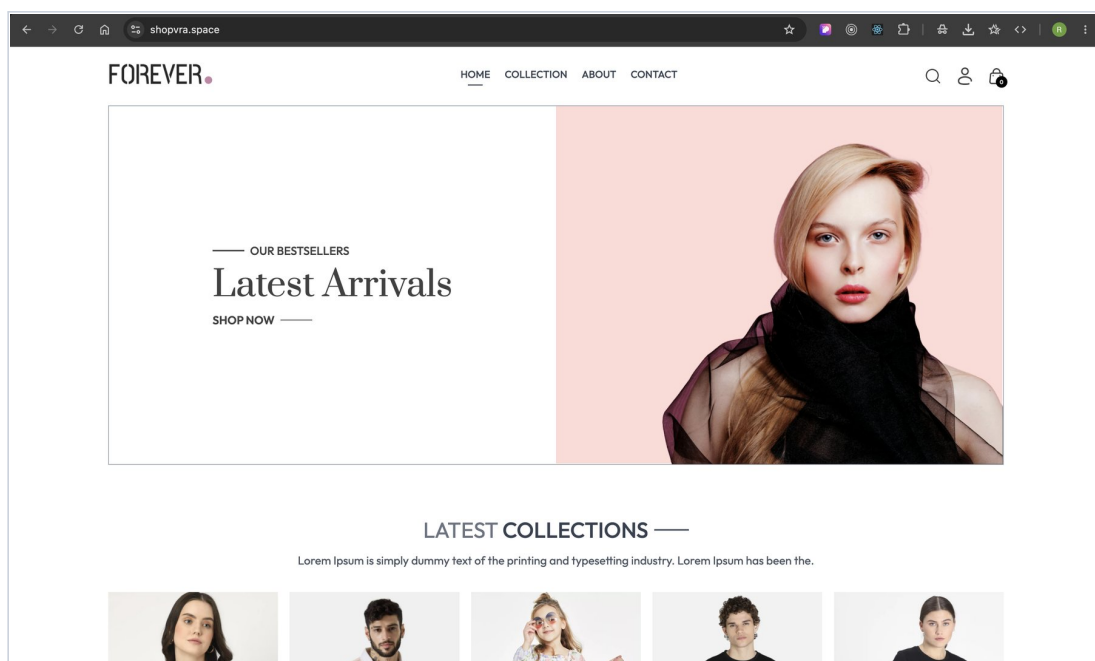
Origin access control is selected. Copy policy gives you the exact statement for this distribution.



Bucket policy with the `AllowCloudFrontServicePrincipal` statement. The old public “Statement1” is removed in Section 4.

13 Step 13 — Test the live site

When **Last modified** changes from “Deploying” to a date (about 5–15 minutes), open <https://shopvra.space>, <https://www.shopvra.space> and a deep link such as `/collection`, then refresh it.



The store live at <https://shopvra.space> over HTTPS.

Done

Frontend is live on your own domain with a valid certificate, served globally by CloudFront.

SECTION 4

Security clean-up for the S3 bucket

The site works, but the bucket still allows public reads from the old static-website setup. Close that so visitors can only reach your files through CloudFront.

1. **S3 → bucket → Permissions → Bucket policy → Edit.** Delete the whole "Sid": "Statement1" block (the one with "Principal": "*"). Keep only the CloudFront statement and save.
2. **Permissions → Block public access → Edit → tick Block all public access → Save → type confirm.**
3. **Properties → Static website hosting → Edit → Disable → Save.**
4. Reload <https://shopvra.space>. It should still work, now served only through CloudFront.

The final bucket policy should look like this

```
{
  "Version": "2008-10-17",
  "Id": "PolicyForCloudFrontPrivateContent",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": { "Service": "cloudfront.amazonaws.com" },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::ecommerce-web-app-084767318696/*",
      "Condition": {
        "ArnLike": {
          "AWS:SourceArn": "arn:aws:cloudfront::084767318696:distribution/E1P27P6JX08L71"
        }
      }
    }
  ]
}
```

If something breaks

If you see an **AccessDenied** XML page after this, the CloudFront statement is missing or has the wrong distribution ID. Copy the policy again from CloudFront → Origins → Edit.

SECTION 5

Deploying future updates

After the first setup, publishing a new version takes three commands. Run them from your React project folder (PowerShell on Windows or Terminal on macOS) with the AWS CLI configured (`aws configure`).

```
# 1. Build the production bundle (Vite -> dist/, Create React App -> build/)
npm run build

# 2. Upload new files and delete ones that no longer exist
aws s3 sync dist/ s3://ecommerce-web-app-084767318696 --delete

# 3. Clear CloudFront's cache so visitors get the new version
aws cloudfront create-invalidation --distribution-id E1P27P6JX08L71 --paths "/*"
```

Command	What it does
<code>npm run build</code>	Creates optimized static files. Environment variables (<code>VITE_*</code> / <code>REACT_APP_*</code>) are baked in at this point, so set them before building.
<code>aws s3 sync ... --delete</code>	Uploads only changed files and removes old ones from the bucket so stale bundles don't pile up.
<code>create-invalidation "/*"</code>	Tells every CloudFront edge to fetch fresh copies. Usually finishes in 1-3 minutes.

Optional: one-click deploy script

Save this as `deploy.ps1` (Windows) in the project root and run `./deploy.ps1`:

```
npm run build
if ($LASTEXITCODE -ne 0) { exit 1 }
aws s3 sync dist/ s3://ecommerce-web-app-084767318696 --delete
aws cloudfront create-invalidation --distribution-id E1P27P6JX08L71 --paths "/*"
Write-Host "Deployed to https://shopvra.space"
```

Or add it to `package.json` and run `npm run deploy`:

```
"scripts": {
  "deploy": "npm run build && aws s3 sync dist/ s3://ecommerce-web-app-084767318696 --delete && aws cloudfront create-invalidation --distribution-id E1P27P6JX08L71 --paths \"/*\""
}
```

Cost note

Invalidations are free for the first 1,000 paths per month, and `/*` counts as one path. Vite adds content hashes to JS/CSS file names, so only `index.html` really needs refreshing.

SECTION 6

Troubleshooting

Common symptoms and what fixes them.

Symptom	Likely cause	Fix
AccessDenied XML page	Bucket policy doesn't allow CloudFront	Copy the OAC policy from CloudFront → Origins → Edit into the bucket policy.
Blank page / 404 on JS files	Wrong base path in the build	Vite: base: '/' in vite.config. CRA: "homepage": "/" in package.json.
403/404 when refreshing /collection	No SPA error responses	Add 403 and 404 → /index.html → 200 (Step 11).
Old version still showing	CloudFront cache	Run the invalidation command, then hard-refresh (Ctrl/Cmd+Shift+R).
Site can't be reached	DNS still spreading, or no A records	Check Route 53 has A/AAAA records; try mobile data; wait 10–30 min.
Certificate not in dropdown	Wrong region or not Issued	Request it in us-east-1 and wait for status Issued.
API calls blocked (mixed content)	Backend is plain http://	Give the backend HTTPS: see Section 7.

SECTION 7

Next: the backend at `https://api.shopvra.space`

Browsers block calls from the HTTPS site to a plain `http://` backend, so the Spring Boot API needs HTTPS too. Reuse the same certificate because it covers `*.shopvra.space`.

	A. CloudFront in front of Beanstalk	B. Load balancer + HTTPS listener
Monthly cost	Free (CloudFront Free plan)	About \$16–18 for the Application Load Balancer
Scaling / health checks	Basic	Built-in auto scaling and health checks
Best for	Learning, portfolio, low traffic	Standard production setup

Option A: CloudFront in front of Beanstalk

1. CloudFront → Create distribution → Free plan → domain **shopvra.space** → **Add a subdomain: api.**
2. Origin type: **Elastic Load Balancer / Other** → your Beanstalk URL (e.g. `ecom-env.eba-xxxx.us-east-1.elasticbeanstalk.com`), protocol **HTTP only**.
3. Viewer protocol policy: **Redirect HTTP to HTTPS**. Allowed methods: **GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE**.
4. Cache policy: **CachingDisabled**. Origin request policy: **AllViewerExceptHostHeader** (passes Authorization headers and query strings).
5. Select the existing **shopvra.space** certificate → Create → **Route domains to CloudFront**.

Option B: Load balancer with an HTTPS listener

1. Beanstalk → Configuration → Capacity → Environment type: **Load balanced**.
2. Load balancer → Listeners → **Add listener**: port **443**, protocol **HTTPS**, certificate **shopvra.space** (same region, us-east-1).
3. Route 53 → Create record **api** → A, Alias → **Application/Classic Load Balancer** → pick the environment's load balancer.

Then connect the two (either option)

```
# React: .env.production
VITE_API_URL=https://api.shopvra.space
// Spring Boot: allow the frontend origins (CORS)
.allowedOrigins("https://shopvra.space", "https://www.shopvra.space")
```

Your decision

Keep `SERVER_PORT=5000` in Beanstalk environment properties, then rebuild and redeploy the frontend (Section 5). **Which one would you like to set up?** Option A (free, CloudFront) is ideal while learning; Option B (~\$16–18/month) is the standard production pattern.